

Modelling with Classes

Adapted from:
Timothy Lethbridge and Robert Laganieri,
Object-Oriented Software Engineering –
Practical Software Development using UML and Java, 2005
(chapter 5)

5.1 What is UML?

The Unified Modelling Language is a standard graphical language for modelling object oriented software [BRJ99,JBR99,RJB99]

- At the end of the 1980s and the beginning of 1990s, the first object-oriented development processes appeared
- The proliferation of methods and notations tended to cause considerable confusion
- Two important methodologists Rumbaugh and Booch decided to merge their approaches in 1994.
 - They worked together at the Rational Software Corporation
- In 1995, another methodologist, Jacobson, joined the team
 - His work focused on use cases
- In 1997 the Object Management Group (OMG) started the process of UML standardization

UML diagrams

- Class diagrams
 - describe classes and their relationships
- Object diagrams
 - show sets of objects and their relationships
- Use case diagrams
 - show sets of use cases and actors and their relationships
- Interaction (sequence and communication) diagrams
 - show the behaviour of systems in terms of how objects interact with each other
- State diagrams and activity diagrams
 - show how systems behave internally
- Component and deployment diagrams
 - show how the various components of systems are arranged logically and physically

What constitutes a good model?

Ideally, a model should

- use a standard notation
- be understandable by clients and users
- lead software engineers to have insights about the system
- provide abstraction

Models are used:

- to help create designs
- to permit analysis and review of those designs
- as the core documentation describing the system.

The UML provides a standard notation for OOSE models.

- The UML models permit you to specify the structure and (to a lesser extend) the behaviour of an OO system.

5.2 Essentials of UML Class Diagrams

The main symbols shown on class diagrams are:

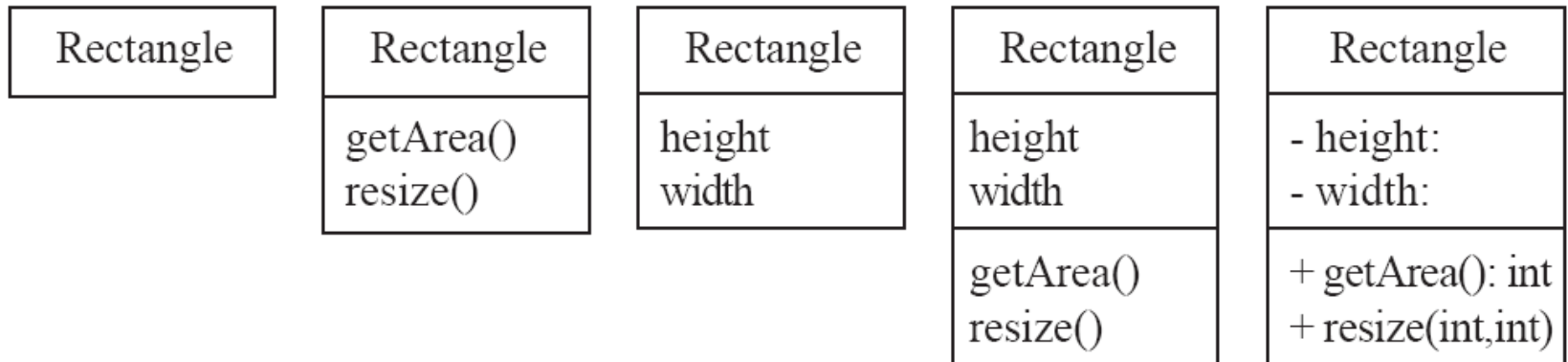
- *Classes*
 - represent the types of objects
- *Associations*
 - describe linkages between instances of classes (objects)
- *Attributes*
 - are simple data found in classes and their instances
- *Operations*
 - represent the functions performed by the classes and their instances
- *Generalizations*
 - group classes into inheritance hierarchies

Classes

A class is simply represented as a box with the name of the class inside

- The diagram may also show the attributes and operations
- The complete signature of an operation is:

`operationName(parameterName: parameterType ...): returnType`



Class attributes and operations

Visibility options for attributes and operations:

- private (-)
 - public (+)
 - protected (#)
 - package or implementation (no icon)
-
- An abstract class is rendered with its name in *italics*.
 - Abstract operations names are also shown in *italics*.

ClassName

- privateAttr : int = 0
protectedAttr : String
+ publicAttr : Long
implAttr : String

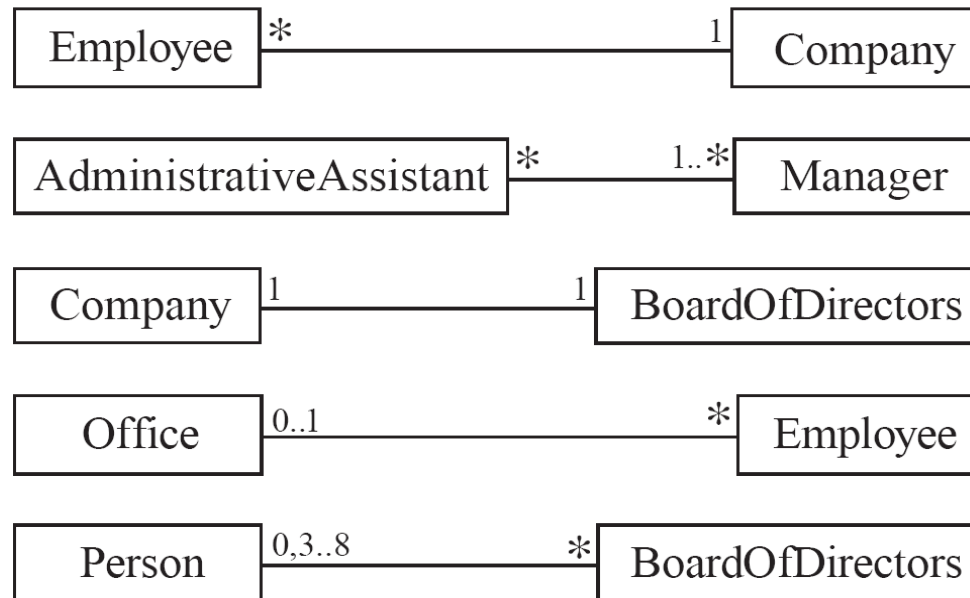
op1(param : String)
+ op2()

AbstractClassName

5.3 Associations and Multiplicity

An *association* is a structural relationship; it is used to show how two classes are related to each other

- An association is rendered as a solid line (possibly directed).
- Symbols indicating *multiplicity* can be shown at each end of the association.
- It is possible but *not recommended* to leave multiplicities undefined.

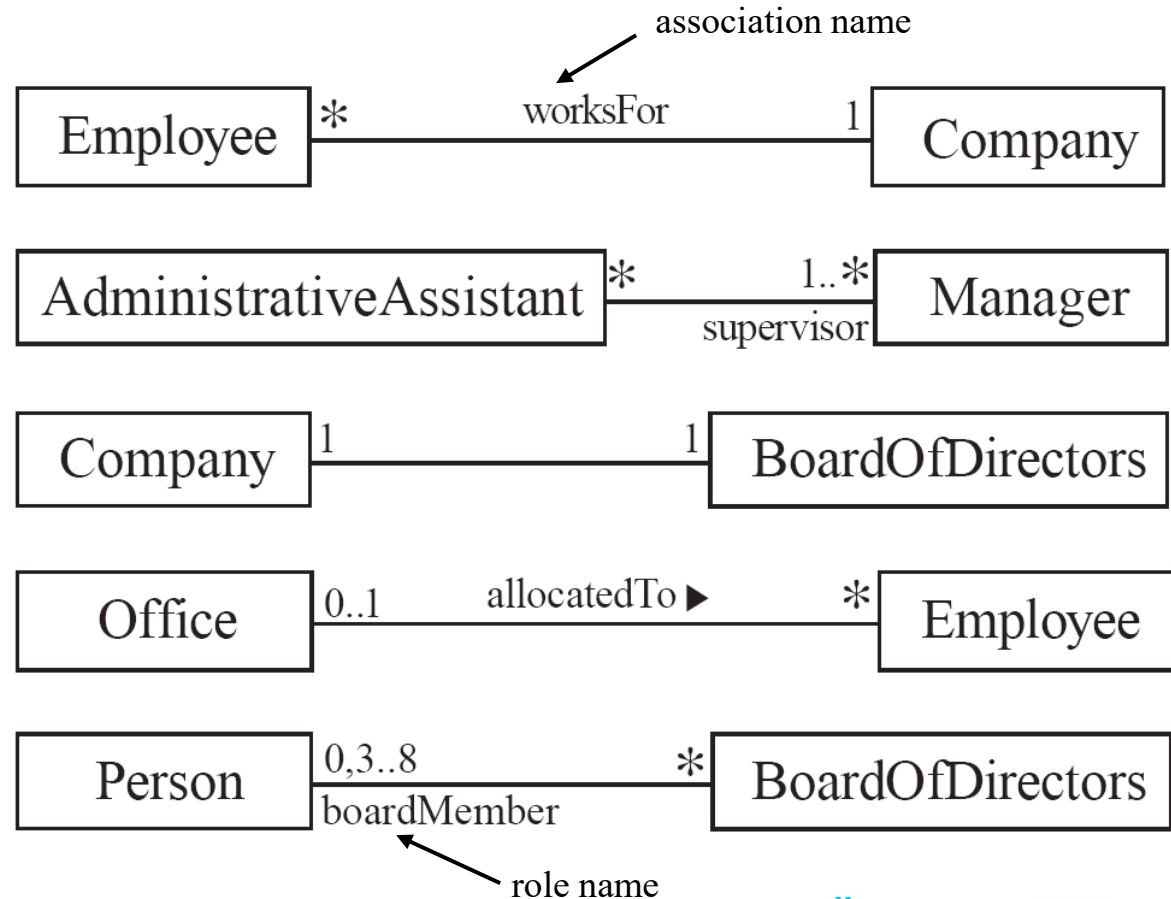


Labelling associations

- Each association can be labelled, to make explicit the nature of the association

Two types of (optional) labels:

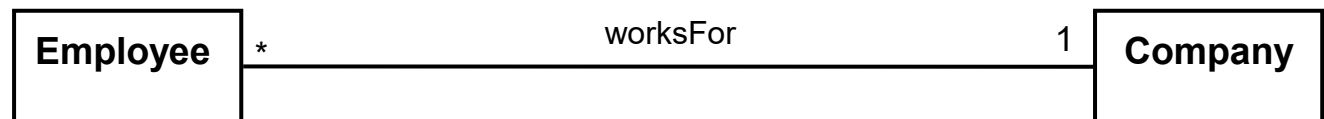
- association names**
 - placed near the middle of the association
 - expressed as verbs or verb phrases
 - '►' can clarify the 'direction' of an association
- role names**
 - attached to either or both ends of the association



Analyzing and validating associations

- **Many-to-one**

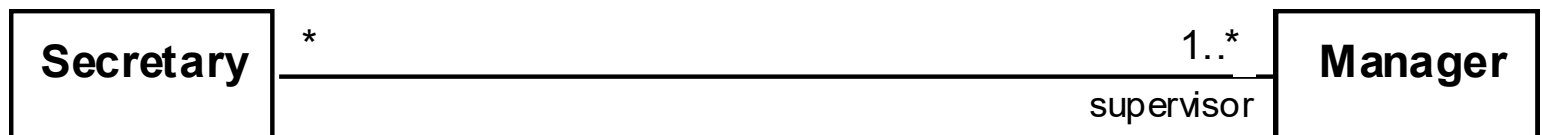
- A company has many employees,
- An employee can only work for one company.
 - This company will not store data about the moonlighting activities of employees!
- A company can have zero employees
 - E.g. a ‘shell’ company
- It is not possible to be an employee unless you work for a company



Analyzing and validating associations

- **Many-to-many**

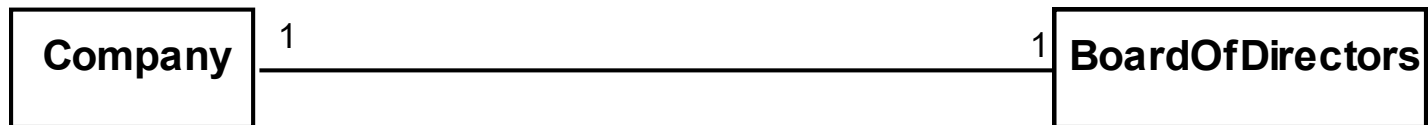
- A secretary can work for many managers
- A manager can have many secretaries
- Managers can have a group of secretaries
- Some managers might have zero secretaries



Analyzing and validating associations

- **One-to-one (less common)**

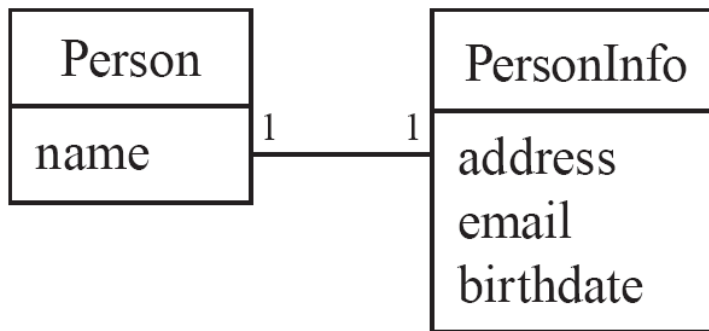
- For each company, there is exactly one board of directors
- A board is the board of only one company
- A company must always have a board
- A board must always be of some company



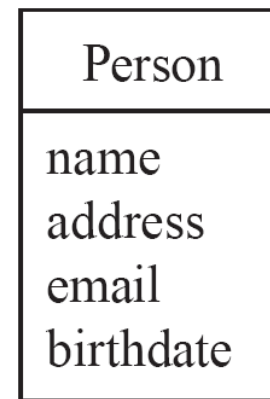
Analyzing and validating associations

Avoid unnecessary one-to-one associations

Avoid this

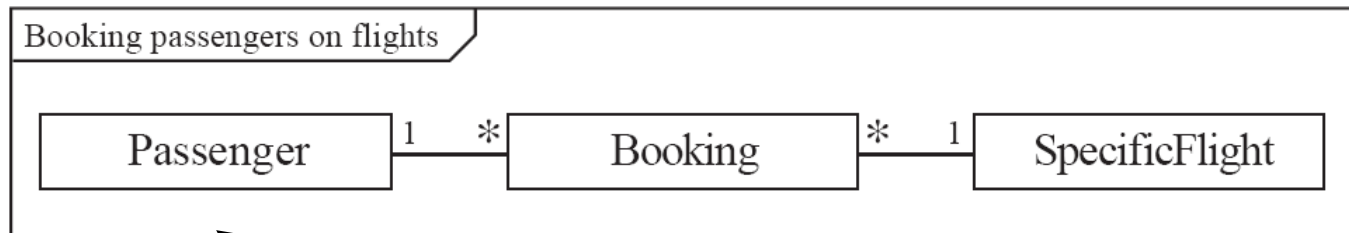


do this



A more complex example

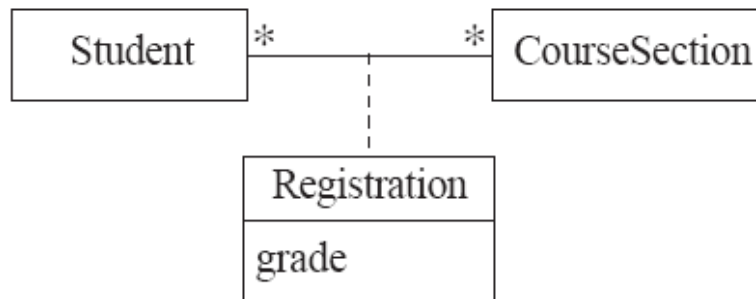
- A booking is always for exactly one passenger
 - no booking with zero passengers
 - a booking could *never* involve more than one passenger.
- A Passenger can have any number of Bookings
 - a passenger could have no bookings at all
 - a passenger could have more than one booking



- This diagram has a **frame** around it (with a **label** 'Booking passengers on flights')
- This is an UML 2.0 feature
- Any UML 2.0 diagram may use frames

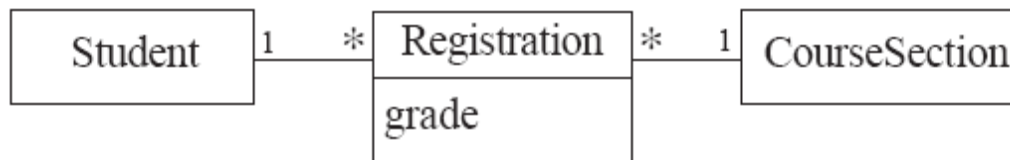
Association classes

- Sometimes, an attribute that concerns two associated classes cannot be placed in either of the classes. The solution is to create an *association class* (e.g. Registration) to hold that attribute (e.g. grade).
- The following two diagrams are semantically equivalent:



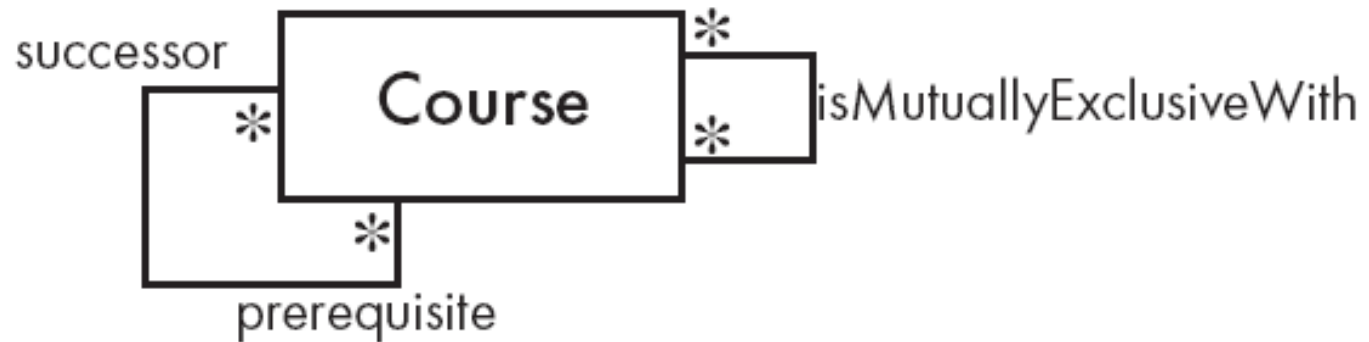
Registration is an *association class*.

Aside from being attached to an association, an association class is no different from any other class.



Reflexive associations

- It is possible for an association to connect a class to itself.
 - A course can require prerequisite courses.
 - Two courses are mutually exclusive if they cover (nearly) the same material.



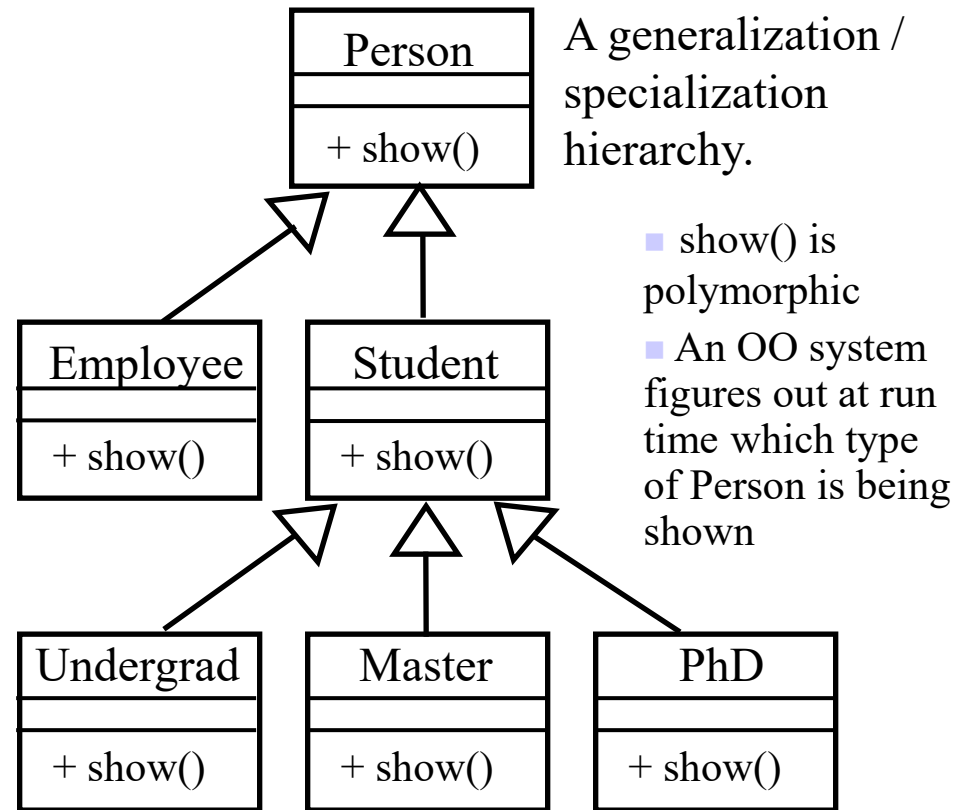
Directionality in associations

- **Associations are by default *bi-directional*.**
- **It is possible to limit the navigability of an association – to make it *unidirectional* - by adding an arrow at one end.**
 - For example, the diagram given below (taken from a calendar application) is designed based on the following assumptions:
 - the user can associate any number of notes with any day;
 - there is no need to determine the day to which a given note belongs.



5.4 Generalization

- The generalization (is-a / is-a-kind-of) relationship, shows an inheritance relationship.
- With inheritance
 - you avoid duplication of attributes and operations by creating a separate super-class that contains common behavior.
 - you can add or change behavior in more specialized derived classes, which “inherit” from the base class.
- The generalization relationship is rendered as a solid line with a small triangle pointing to the super-class.
- Polimorphic behavior:
 - An operation of a sub-class with the same signature as an operation of the super-class overrides the operation of the super-class.

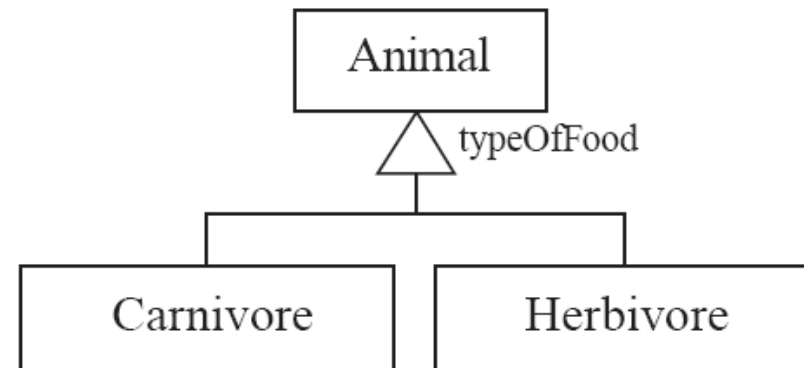
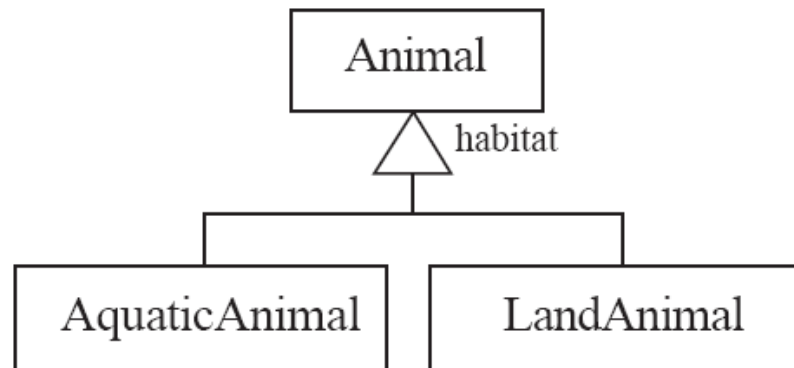


One of the most important rules to adhere is the **is-a rule**:

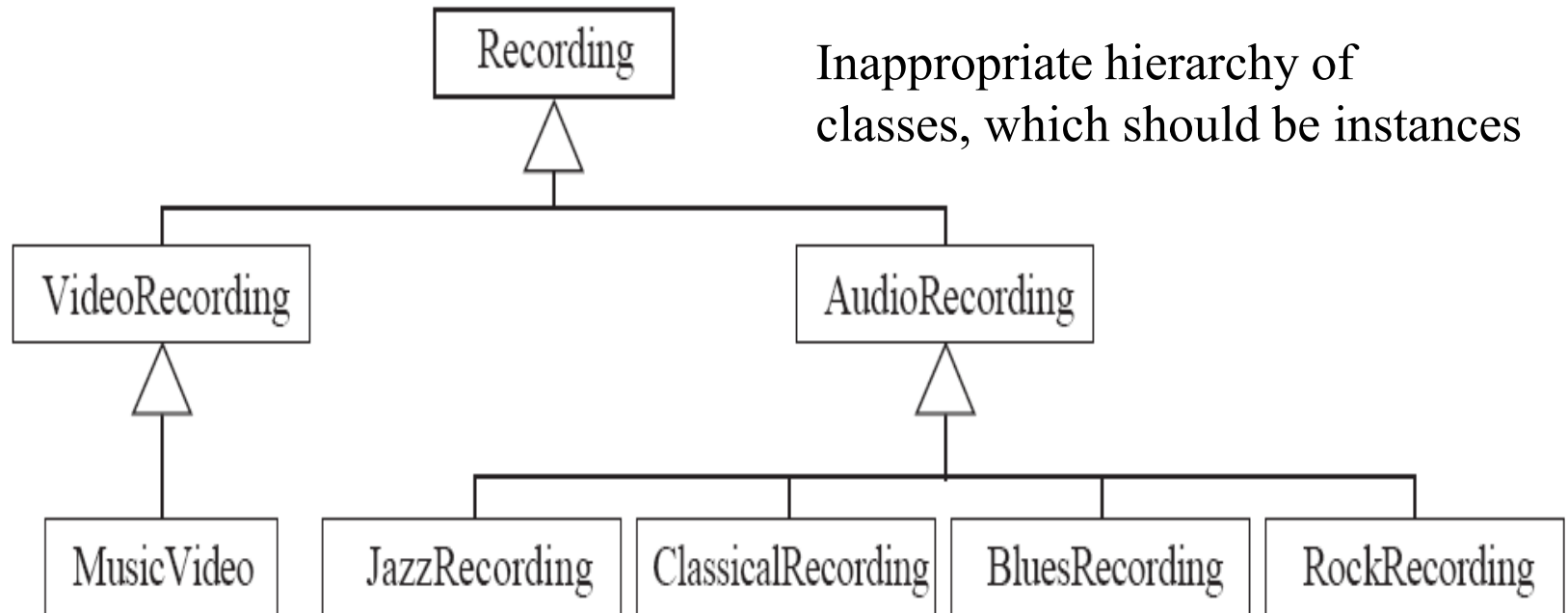
- Class A can only be a valid sub-class of class B if it makes sense, in English, to say ‘an A is a B’.

Generalization sets

- A generalization set is a labeled group of generalizations with a common super-class.
- The label (called discriminator in earlier versions of UML) describes the criteria used to specialize the super-class into two or more sub-classes.
- The label of a generalization set (e.g. habitat, or typeOfFood) will typically be an attribute that has a different value in each sub-class.

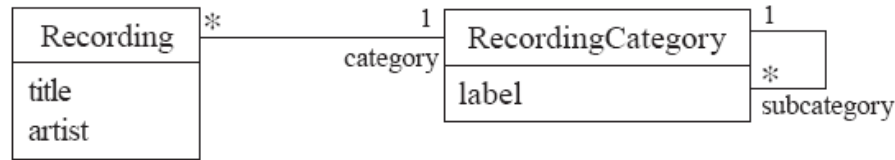


Avoiding unnecessary generalizations

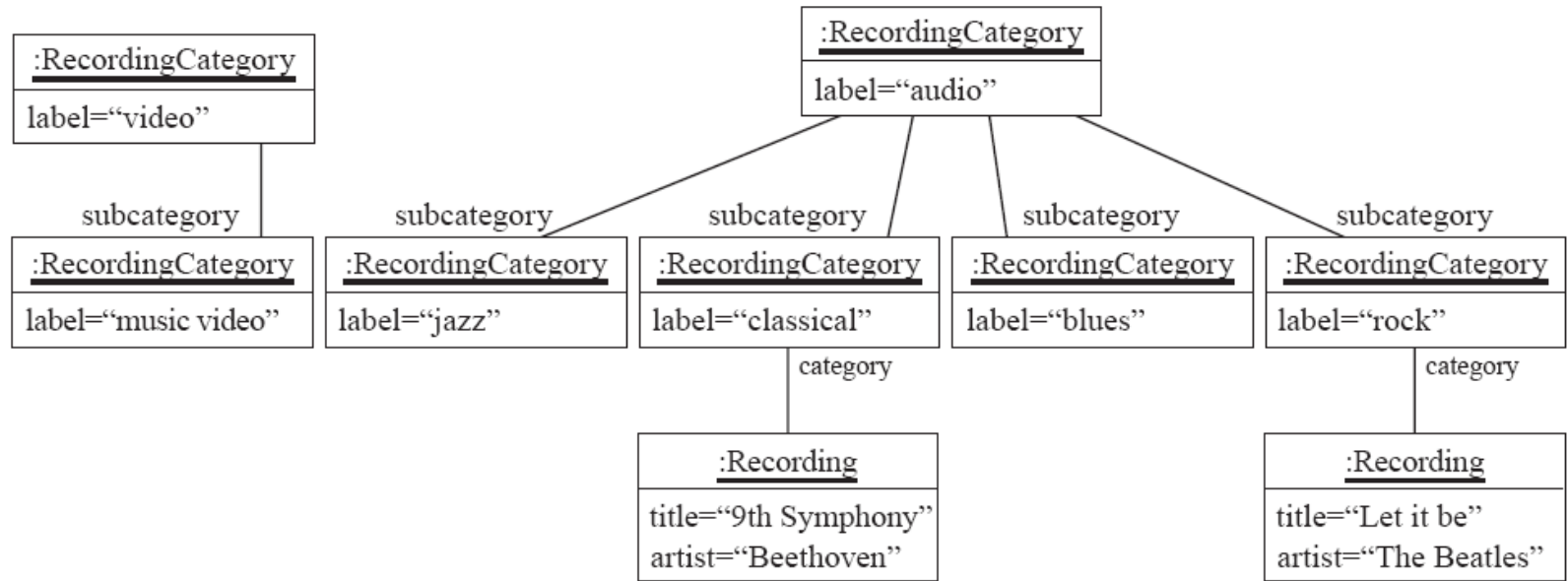


- It makes sense to say ‘an AudioRecording is a Recording’, etc
- However, to justify the existence of each class there must be some operation that will be done differently in that class.
- Most of the classes in this hierarchy behave the same. For example, JazzRecording, ClassicalRecording, BluesRecording and RockRecording would not differ with regard to how they are sold, nor with regard to what kind of information clients can find about them.

Avoiding unnecessary generalizations (cont)



(a)



(b)

Improved class diagram (a), with a corresponding object (instance) diagram (b)

Handling multiple generalization sets

- **Modeling problem:**

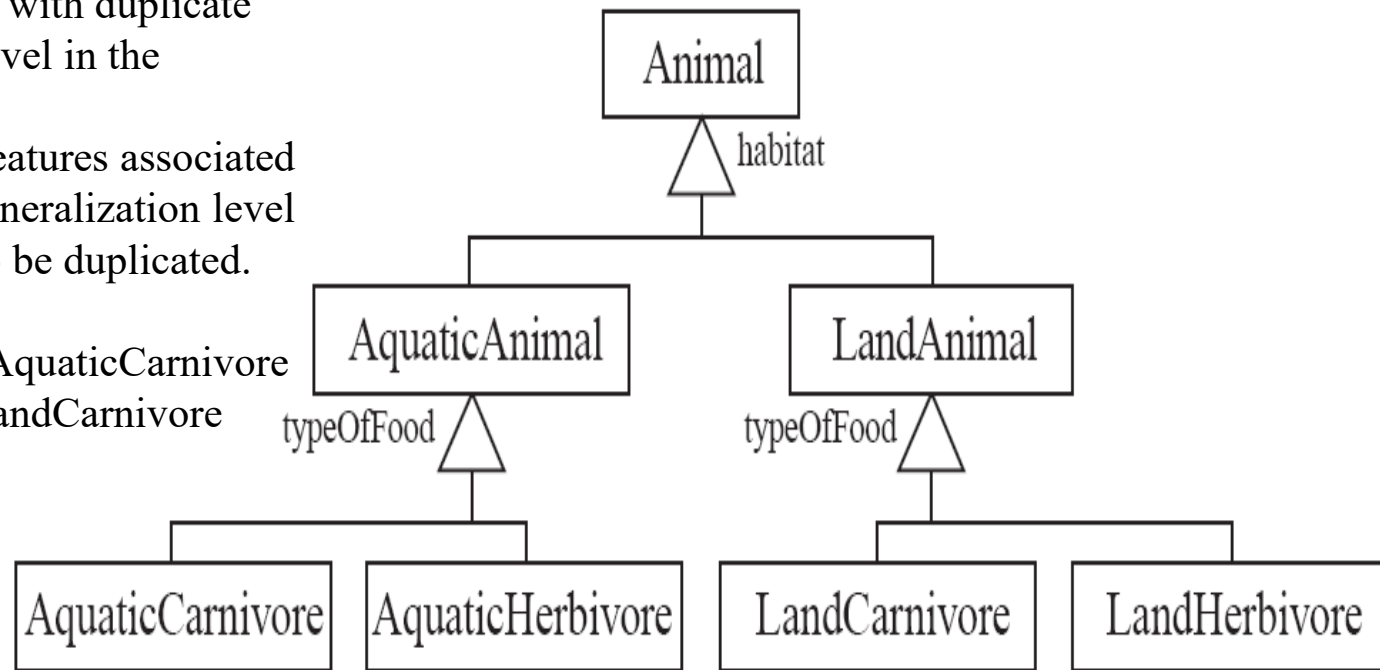
- You want to find a way to enable animals to have all possible combinations of habitat and type of food (e.g., you want to represent aquatic carnivores such as sharks)

- **One possible solution** is to create a higher-level generalization (habitat), and then have generalization sets with duplicate labels at a lower level in the hierarchy

- **Drawback:** all features associated with the second generalization level would also have to be duplicated.

For example:

- prey of an AquaticCarnivore
- prey of a LandCarnivore



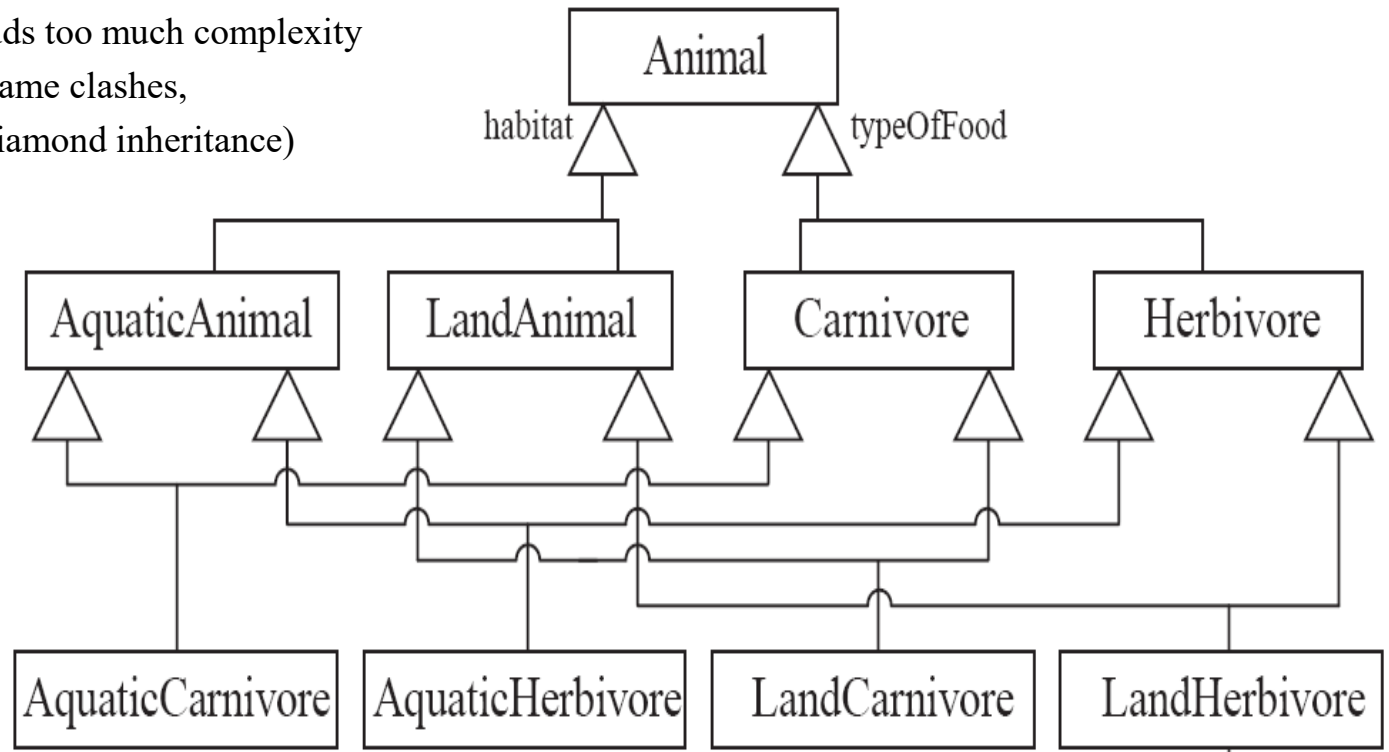
Handling multiple generalization sets

- **Solution based on multiple inheritance**

- Advantage: avoids duplication of features (e.g. prey only associated to Carnivore)

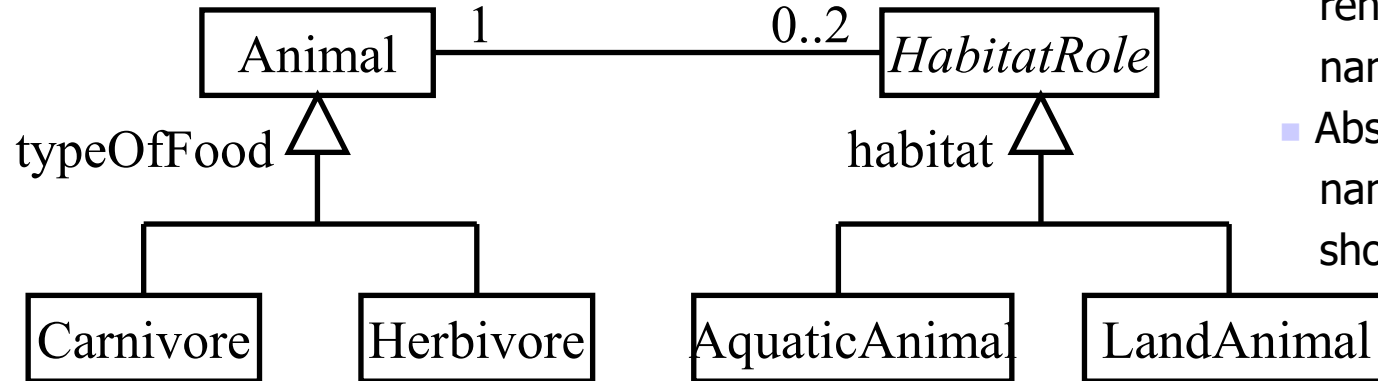
- Disadvantages:

- even more classes and generalizations
 - multiple inheritance generally adds too much complexity (name clashes, diamond inheritance)



Handling multiple generalization sets

- A superior solution (based on the Player-Role design pattern)



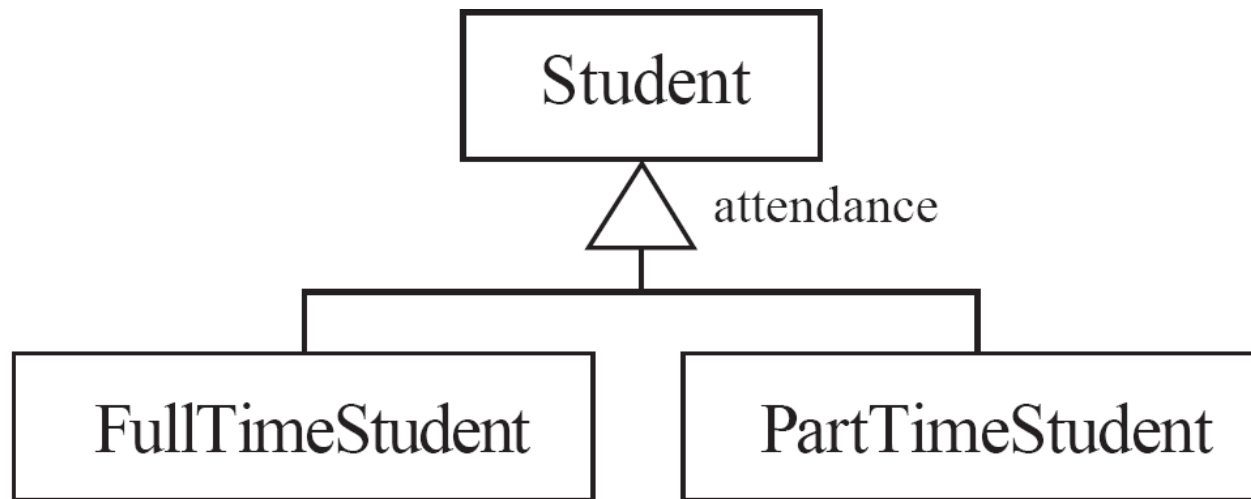
In the UML:

- An abstract class is rendered with its name in *italics*.
- Abstract operations names are also shown in *italics*.

- The template of the Player-Role design pattern consists of a Player class associated with an abstract Role class; the Role class is the super-class of a set of possible concrete roles.
 - A *role* is a particular set of features associated with an object in a particular context
 - An object may *play* different roles in different contexts
- In the diagram given above the roles (AquaticAnimal, LandAnimal) are unlikely to change.

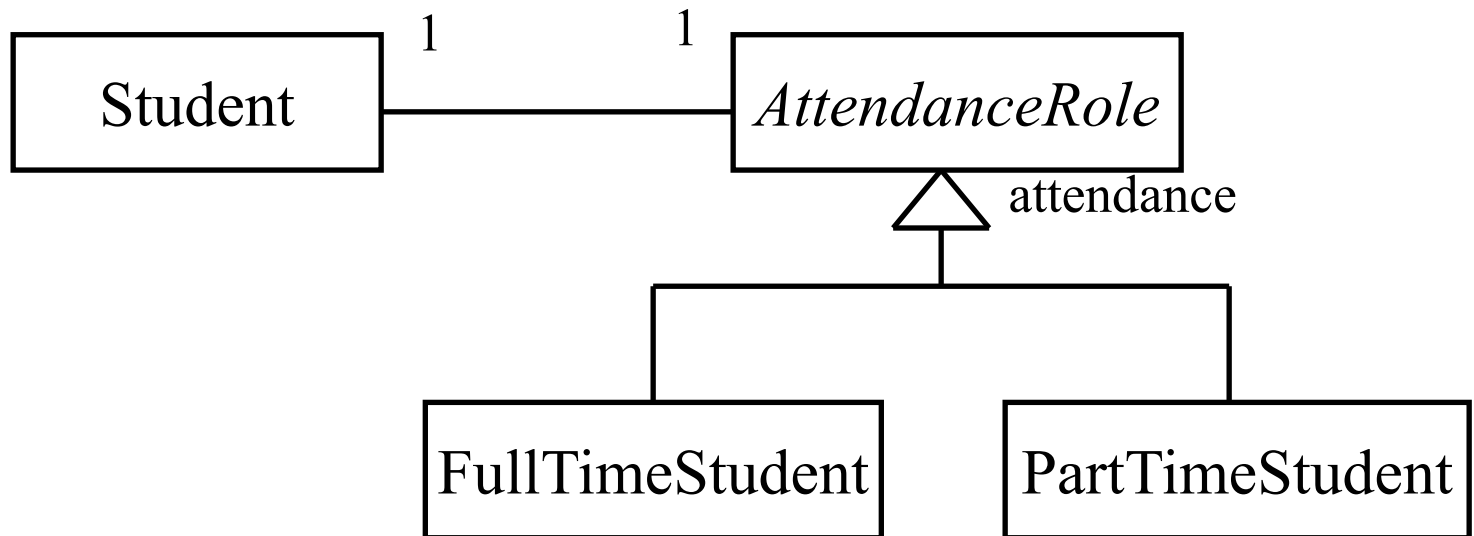
Avoiding having instances change class

- An instance should never need to change class !
 - During his/her studies, the attendance status of a student can change from full-time to part-time and vice versa.
 - You do not want to model this by destroying an instance of PartTimeStudent (FullTimeStudent) and creating an instance of FullTimeStudent (PartTimeStudent). **The diagram given below is a poor model !**



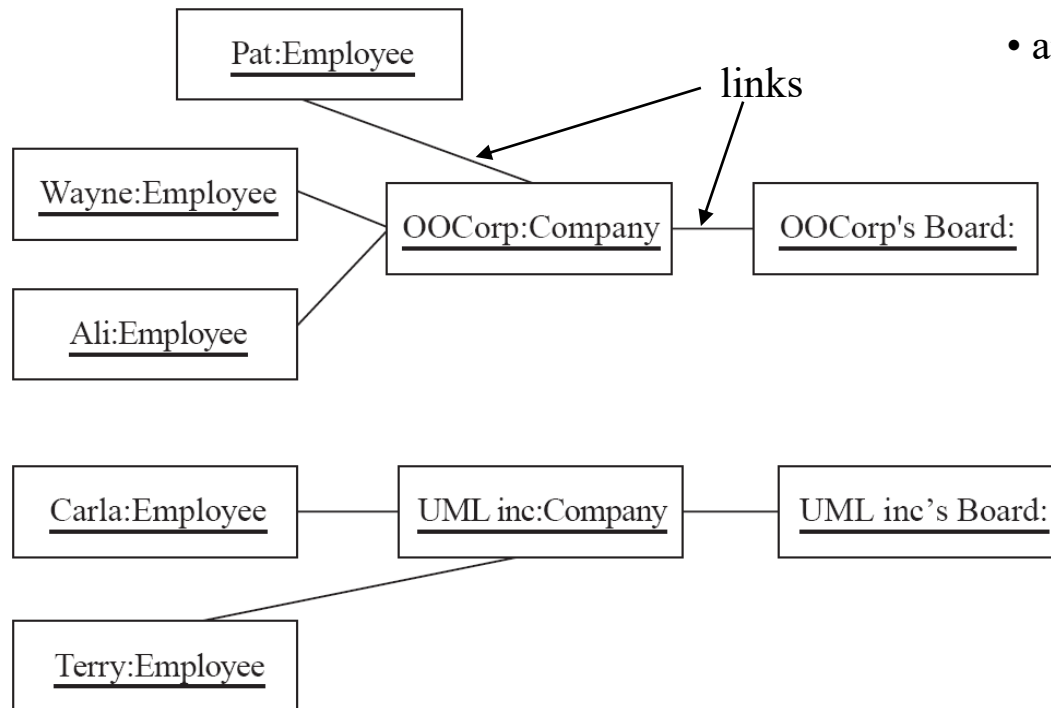
Avoiding having instances change class

- A superior solution (again based on the Player-Role pattern)



5.5 Object Diagrams

- A **link** is an instance of an association
—in the same way that we say an
object is an instance of a class



In the UML an **object** is shown as a rectangle (just like a class), but the name of the object is underlined.

You can represent:

- named objects:
 - objectName : ClassName
- anonymous objects:
 - : ClassName

Object diagram for

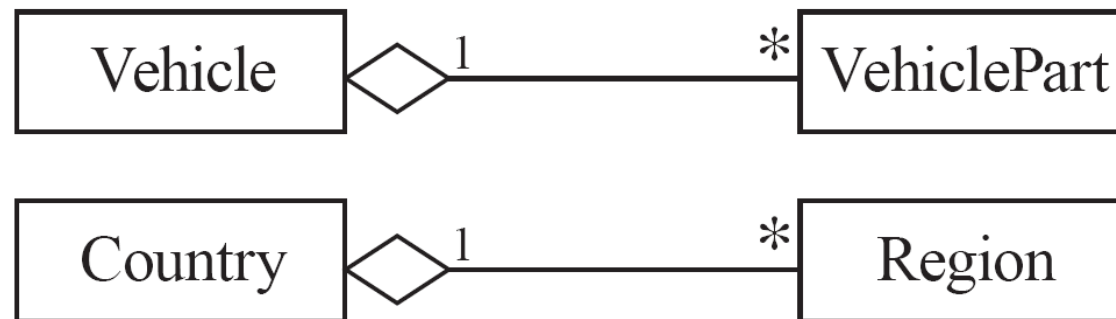
- the one-to-many association between Employee and Company, and
- the one-to-one association between Company and BoardOfDirectors

Associations versus generalizations in object diagrams

- Associations describe the relationships that will exist between *instances* at run time.
 - When you show an instance diagram generated from a class diagram, there will be an instance of *both* classes joined by an association
- Generalizations describe relationships between *classes* in class diagrams.
 - They do not appear in instance diagrams at all.
 - An instance of any class should also be considered to be an instance of each of that class's super-classes

5.6 More Advanced Features: Aggregation

- Aggregations are special associations that represent ‘part-whole’ relationships (the `hasA` or `isPartOf` relationship).
 - The ‘whole’ side is often called the *assembly* or the *aggregate*
 - Graphically, an aggregation is rendered as an association adorned with an open diamond at the ‘whole’ end.



When to use an aggregation

As a general rule, you can mark an association as an aggregation if the following are true:

- You can state that
 - the parts ‘are part of’ the aggregate
 - or the aggregate ‘is composed of’ the parts
- When something owns or controls the aggregate, then they also own or control the parts



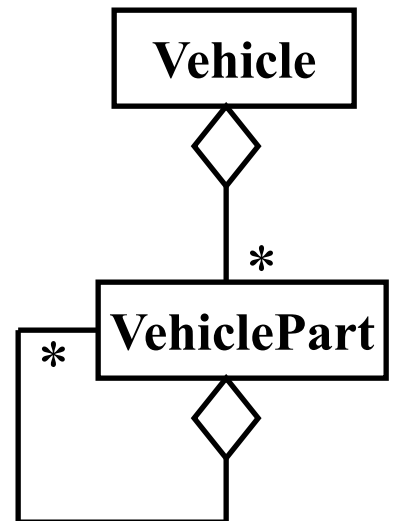
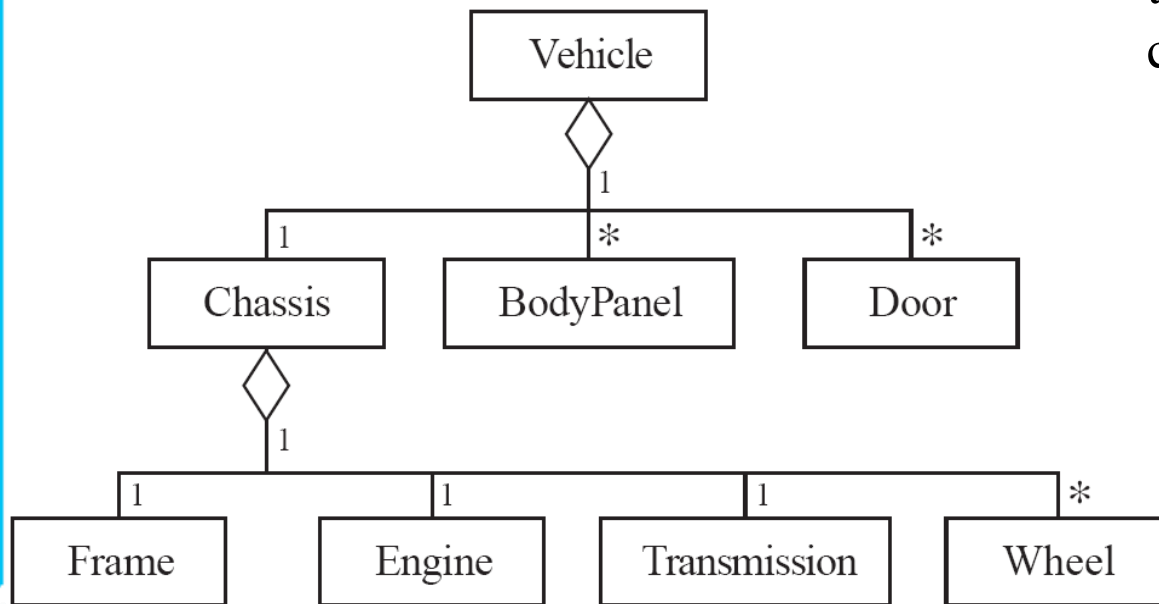
Composition

- A *composition* is a strong kind of aggregation
 - if the aggregate is destroyed, then the parts are destroyed as well



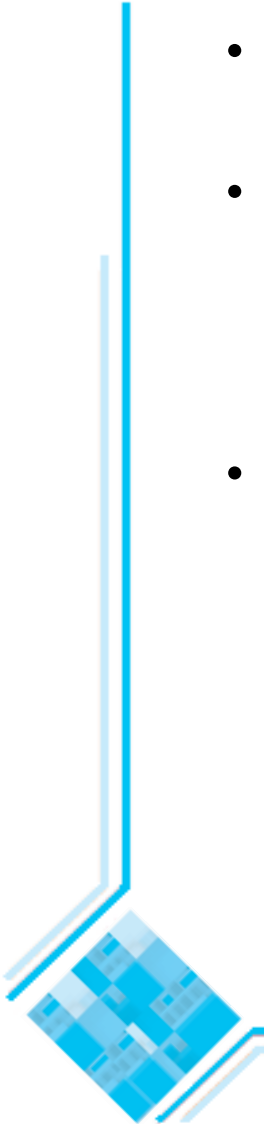
Aggregation hierarchy

This is more flexible a representation. It can easily accommodate new types of vehicles that have different configurations of parts.



Propagation

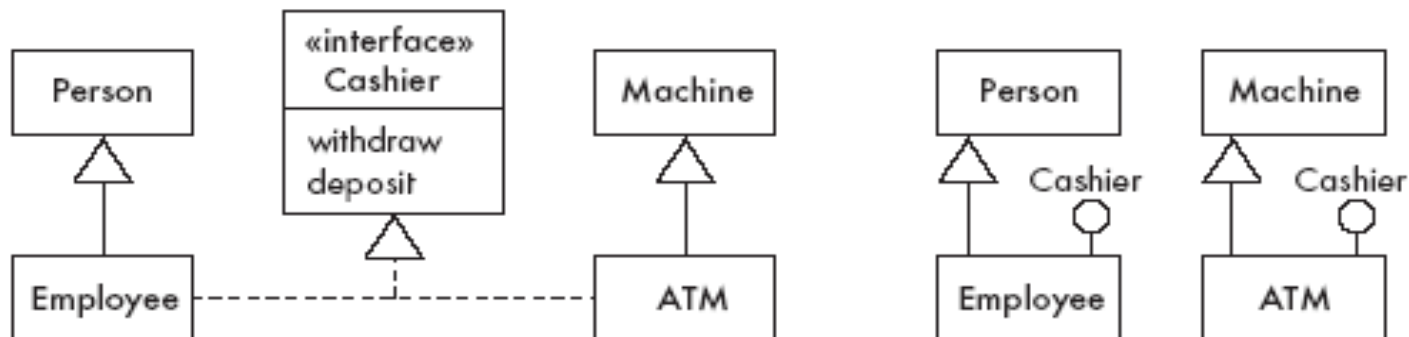
- A mechanism where an operation in an aggregate is implemented by having the aggregate perform that operation on its parts
- At the same time, properties of the parts are often propagated back to the aggregate
 - For example, the weight of an aggregate (e.g. a vehicle) could be obtained by summing the weights of its parts.
- Propagation is to aggregation as inheritance is to generalization.
 - The major difference is:
 - inheritance is an implicit mechanism
 - propagation has to be programmed when required



Interfaces

An interface describes a *portion of the visible behaviour* of a set of objects.

- An *interface* is similar to a class, except it lacks instance variables and implemented methods
- In the UML, the relationship between an interface and the class (or the component) which implements (realizes) it is called a *realization*.
 - A realization is rendered as a dashed line with a small triangle.
- In the UML there are two ways to specify interfaces, both of which are shown in the figure below (where the Cashier interface is implemented by the classes Employee and ATM).



5.9 The Process of Developing Class Diagrams

You can create UML models at different stages and with different purposes and levels of details

- **Exploratory domain model:**
 - Developed in domain analysis to learn about the domain
 - It normally has some classes, associations and attributes that are outside the scope of the system (which are only needed to understand the domain).
 - At this stage you are not concerned with model optimization techniques, such as avoiding multiple inheritance.
- **System domain model:**
 - Models aspects of the domain represented by the system
 - The classes in this model become real software modules and the instances of many of these classes end up being stored persistently in some kind of database.
 - This model can contain less than half the classes of the complete system model.
 - Should be developed to be used independently of particular sets of user interface classes or architectural classes.
- **(Complete) System model:**
 - Includes the system domain model, and it also
 - Adds classes used to build the user interface, the system architecture (clients, servers, layer protocols, etc.), as well as utility classes (that make the system more reusable, more maintainable, etc.)

Suggested sequence of analysis & design activities

- Identify a first set of candidate **classes**
- Add **associations** and **attributes**
- Find **generalizations**
- List the main **responsibilities** of each class
- Decide on specific **operations** that are needed to realize each responsibility
- **Iterate** over the entire process until the model is satisfactory
 - Add or delete classes, associations, attributes, generalizations, responsibilities or operations
 - Identify interfaces, apply design patterns and principles

Don't be disorganized. Don't be too rigid either.

Identifying classes

- When developing a domain model you tend to *discover* classes (based on requirements descriptions, interview notes, or results of brainstorming sessions)
 - Sometimes you may have to invent elements of a domain model: for example you may have to invent a super-class (whose name is not in the domain) in order to obtain a generalization hierarchy.
- When you work on the user interface or the system architecture, you tend to *invent* classes
 - Such classes are needed to solve a particular design problem
- When identifying classes reuse should always be a concern; you can
 - reuse frameworks
 - extend existing systems
 - look at similar systems to obtain useful insights.

A simple technique for discovering domain classes

- Look at a source material such as a description of requirements
- Extract the *nouns* and *noun phrases*
- Eliminate nouns that:
 - are redundant (two names for the same class)
 - represent instances
 - are vague or highly general (for example: ‘the user’s goal’, or ‘the information this application will represent’)
 - not needed in the application
- Pay attention to classes in a domain model that represent *types of users* or other actors
- Make a clear distinction between classes and class attributes

A simple technique for discovering domain classes

Example: Using the description of the Airlines system given below, list the nouns and noun phrases that might end up being classes in a system domain model.

Ootumlia Airlines runs sightseeing flights from Java Valey, the capital of Ootumlia. The reservation system keeps track of passengers who will be flying in specific seats on various flights, as well as people who will form the crew. For the crew, the system needs to track what everyone does, and who supervises whom. Ootumlia Airlines runs several daily numbered flights on a regular schedule. Ootumlia Airlines expects to expand in the future, therefore the system needs to be flexible; in particular, it will be adding a frequent-flier plan.

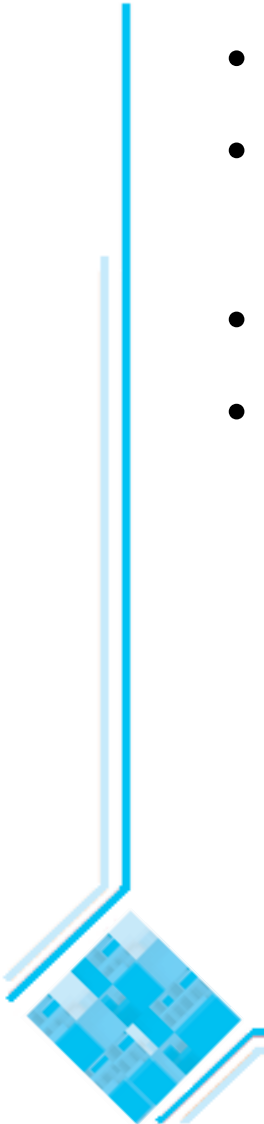
A simple technique for discovering domain classes

Solution:

- Nouns that are put in the initial list of classes include: Flight, Passenger, Employee.
- We choose not to include the other nouns or noun phrases because:
 - ‘Ootumlia Airlines’, ‘Java Valey’, ‘Ootumlia’ are instances
 - ‘Reservation system’ is the name of the system itself
 - ‘SeightSeeing Flight’ rejected in favor of Flight
 - ‘Seat’ appears to be an attribute of Flight
 - ‘Crew’ - this implies the entire crew; we could have created a class ‘CrewMember’, but ‘Employee’ is more flexible
 - ‘Schedule’ describes complex information that is best represented using classes such as Flight, with its associations and attributes.
 - ‘Future’ – this is a noun but is not part of the system to be developed
 - ‘Frequent-flier plan’ is not part of the current scope

Identifying associations and attributes

- Start with classes you think are most **central** and important
- For each of these classes, decide on the clear and obvious data it must contain and its relationships to other classes.
- Work outwards towards the classes that are less important.
- Avoid adding many associations and attributes to a class
 - A system is simpler if it manipulates less information
 - As you add an association or an attribute, make sure it is relevant to the application – that it will be needed to implement some requirement



Tips about identifying and specifying valid associations

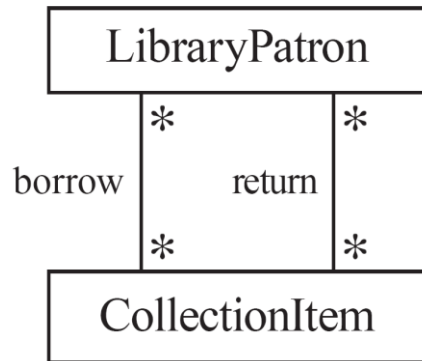
- An association should exist if a class
 - *possesses*
 - *controls*
 - *is connected to*
 - *is related to*
 - *is a part of*
 - *has as part*
 - *is a member of, or*
 - *has as member*

some other class in your model (such information can often be extracted from the requirements document)

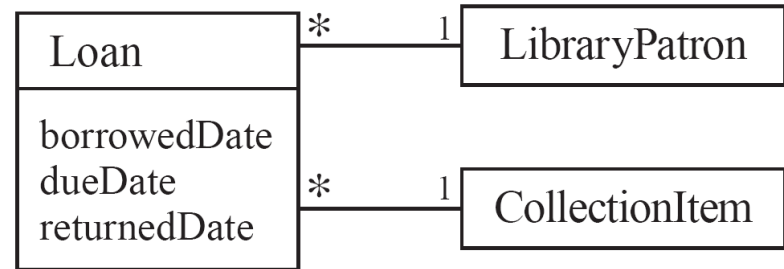
- Specify the multiplicity at both ends
- Label it clearly.

Actions versus associations

- A common mistake is to represent *actions* (or use cases) as if they were associations



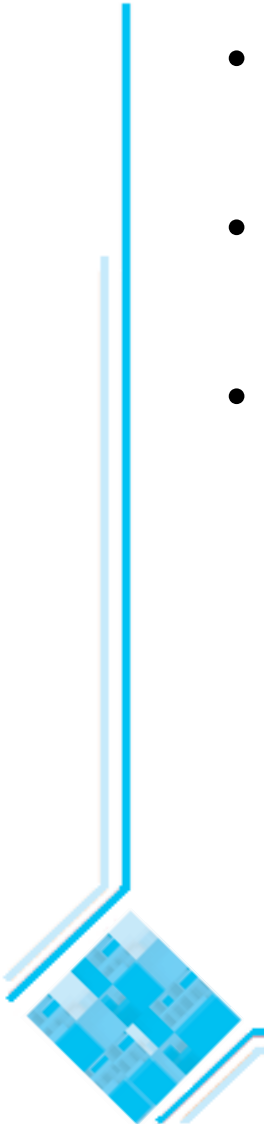
Bad, due to the use of associations that are actions.



Better: the borrow operation creates a Loan and the return operation sets the returnDate attribute

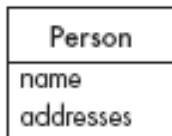
Identifying attributes

- Look for information that must be maintained about each class
- Several nouns rejected as classes, may now become attributes
- An attribute should generally contain a simple value
—E.g. string, number



Tips about identifying and specifying valid attributes

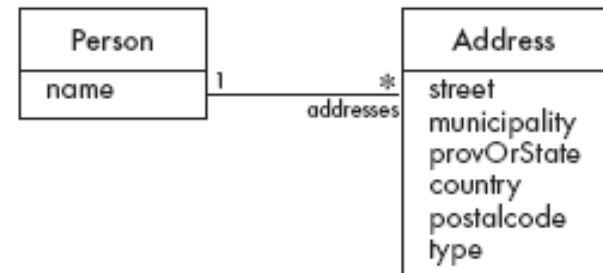
- It is not good to have many duplicate attributes
- If a subset of a class's attributes form a coherent group, then create a distinct class containing these attributes



Bad, due to a plural attribute



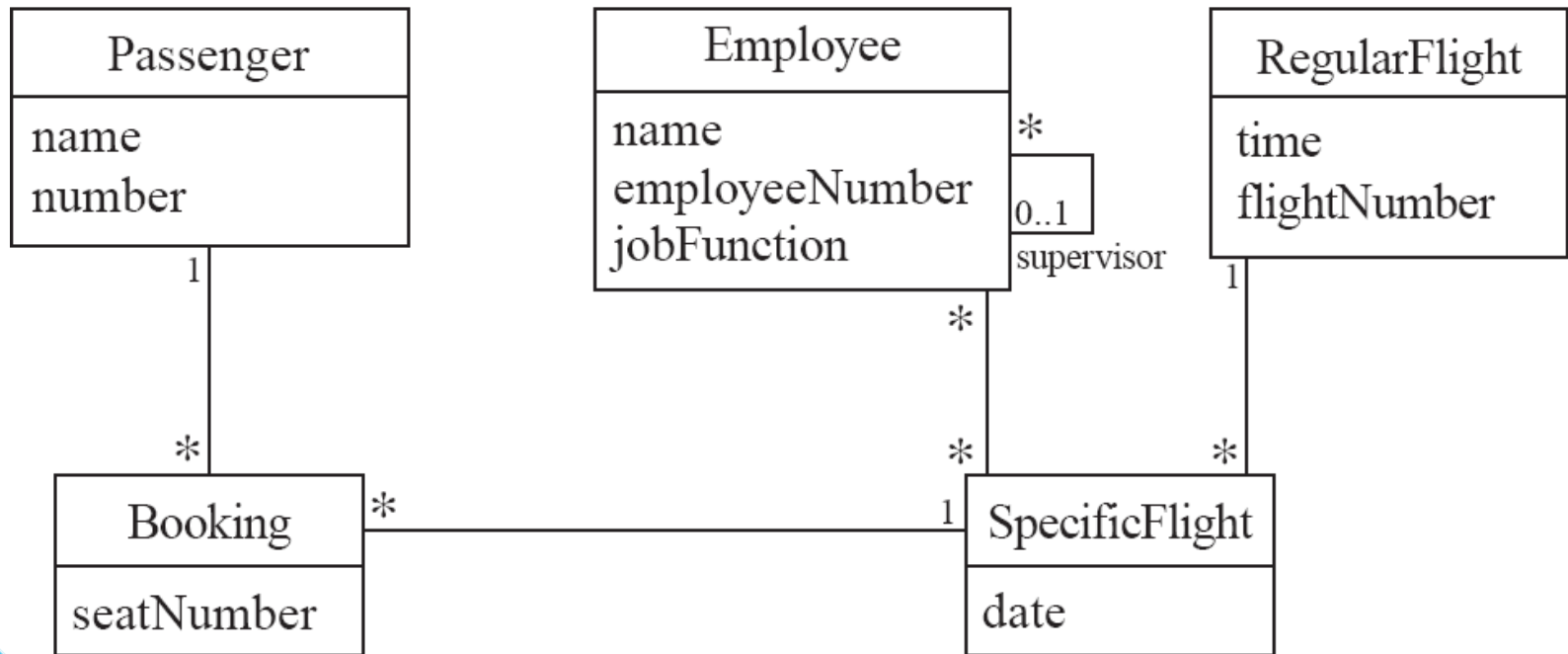
Bad, due to too many attributes, and the inability to add more addresses



Good solution. The type indicates whether it is a home address, business address etc.

Example: attributes and associations for the Airline system

- A central class at which to start identifying associations and attributes is Flight. To avoid redundancy, this is split into: RegularFlight, and SpecificFlight.
- Next it can be studied how the passengers are booked.

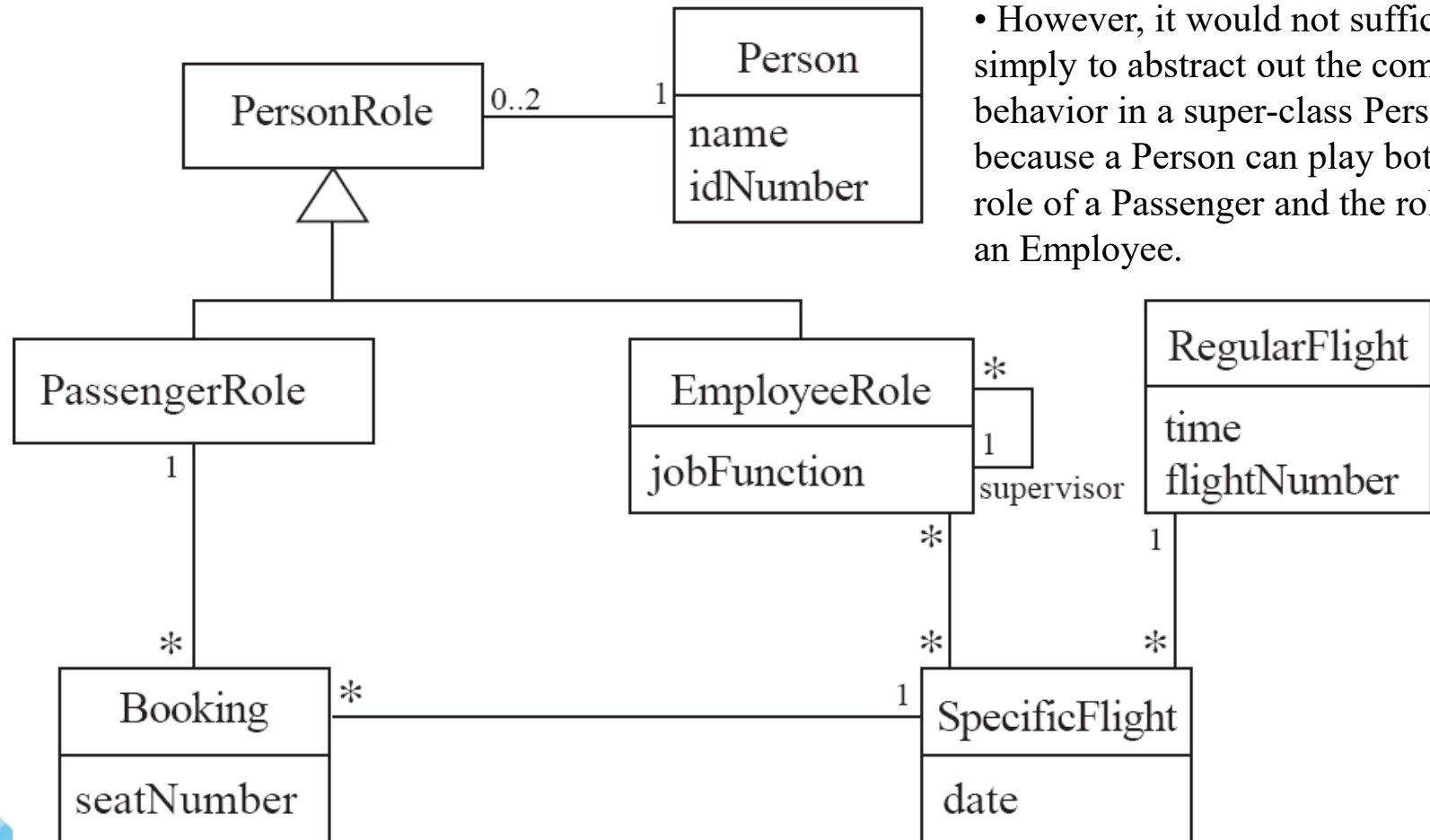


Identifying generalizations and interfaces

- There are two ways to identify generalizations:
 - bottom-up
 - Group together two or more classes that share some common behaviour (attributes, associations or operations) by creating a new super-class (that represents the common behavior)
 - top-down
 - Look for more general classes first, specialize them if needed
- Create an *interface*, instead of a super-class if
 - The classes are very dissimilar except for having a few operations in common
 - One or more of the classes already have their own super-classes
 - You want to limit the operations that can be performed on a variable to just those available in the interface

An example (generalization)

- In the Airline example, the classes Passenger and Employee share the attributes for name and identification number.
- However, it would not suffice simply to abstract out the common behavior in a super-class Person, because a Person can play both the role of a Passenger and the role of an Employee.



Allocating responsibilities to classes

A *responsibility* is something that the system is required to do

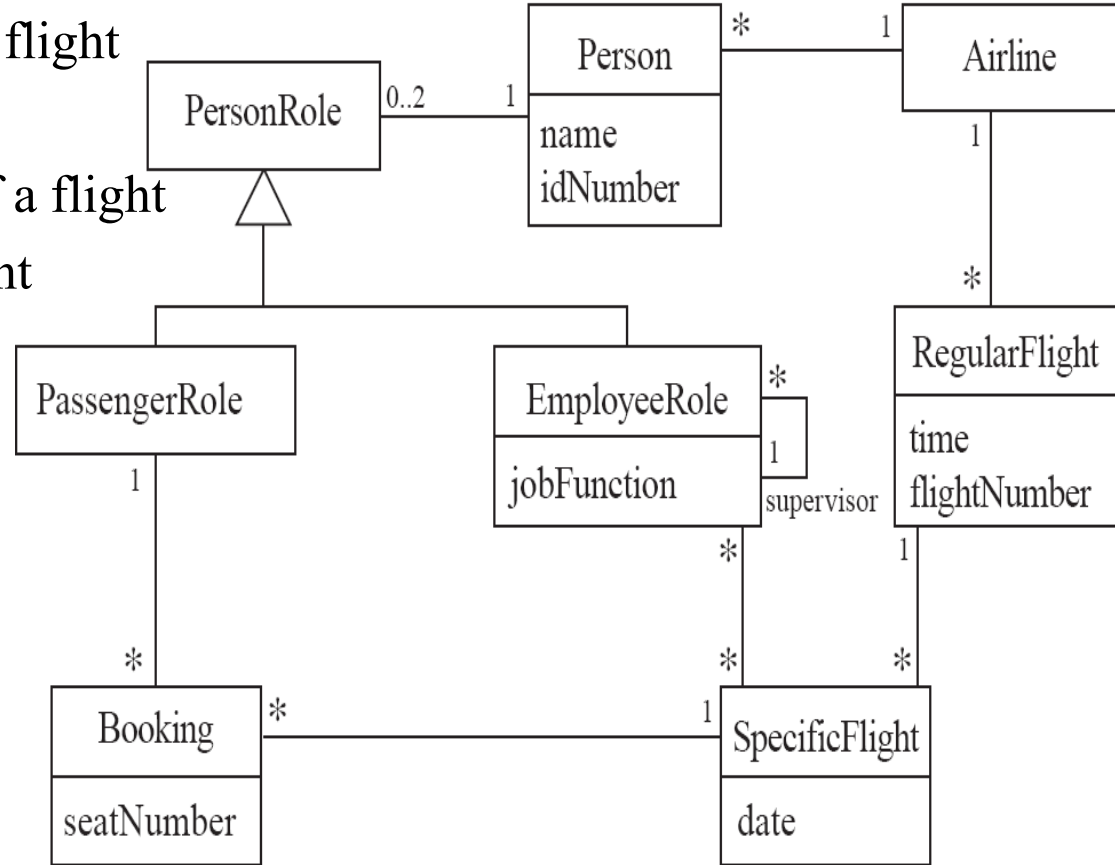
- The main responsibilities are given by the functional requirements.
- Each responsibility must be attributed to one of the classes, although other classes will likely collaborate with it to help perform the task.
 - All the responsibilities of a given class should be *clearly related*.
 - If a class has too many responsibilities, consider *splitting* it into distinct classes
 - If a class has no responsibilities attached to it, then it is probably *useless*
 - When a responsibility cannot be attributed to any of the existing classes, then a *new class* should be created
- To determine responsibilities
 - Perform use case analysis
 - Look for verbs and nouns describing *actions* in the system description

Categories of responsibilities

- Setting and getting the values of attributes
- Creating and initializing new instances
- Loading to and saving from persistent storage
- Destroying instances
- Adding and deleting links of associations
- Copying, converting, transforming, transmitting or outputting
- Computing numerical results
- Navigating and searching
- Other specialized work

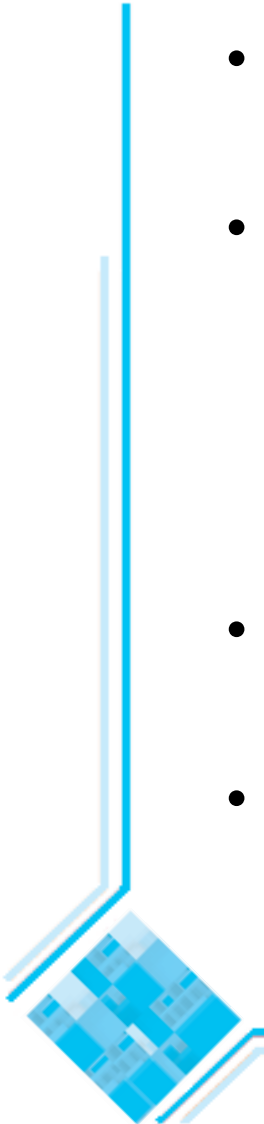
An example (responsibilities)

- Creating a new regular flight
- Searching for a flight
- Modifying attributes of a flight
- Creating a specific flight
- Booking a passenger



Prototyping a class diagram on paper: CRC (Class-Responsibility-Collaboration) cards

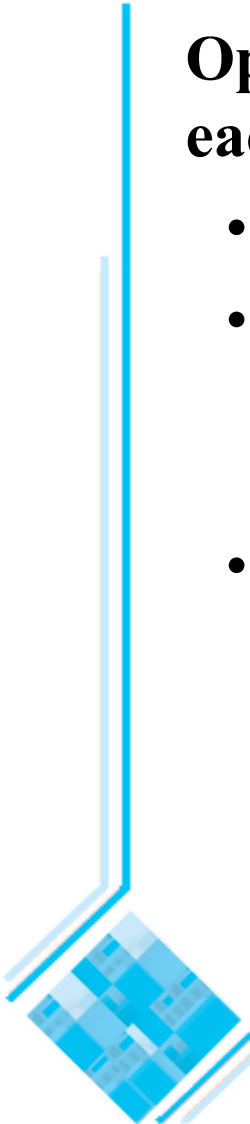
- As you identify classes, you write their names on small cards
- As you identify attributes and responsibilities, you list them on the cards
 - If you cannot fit all the responsibilities on one card:
 - this suggests you should split the class into two related classes.
- Move the cards around on a whiteboard to arrange them into a class diagram.
- Draw lines among the cards to represent associations and generalizations.



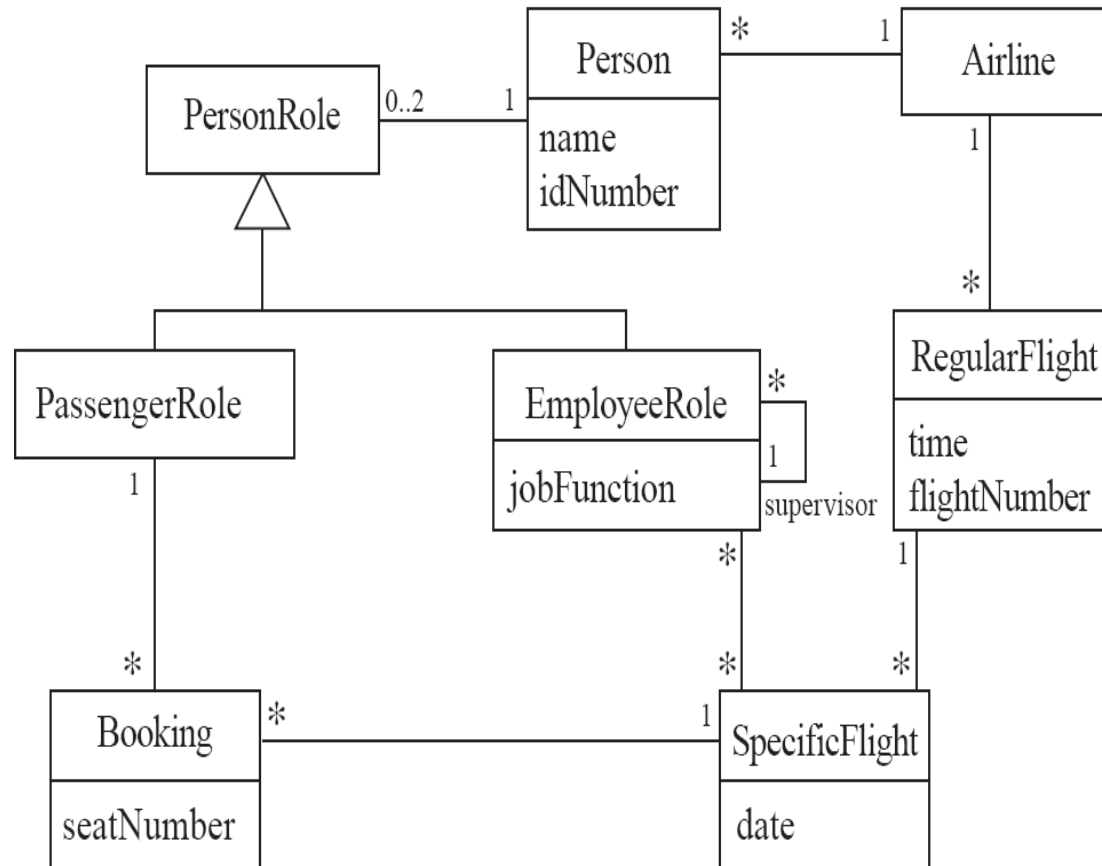
Identifying operations

Operations are needed to realize the responsibilities of each class

- There may be several operations per responsibility
- The main operation that implements a responsibility is normally declared **public**; it becomes part of the interface of the entire system.
- Other methods that collaborate to perform the responsibility must be as private as possible

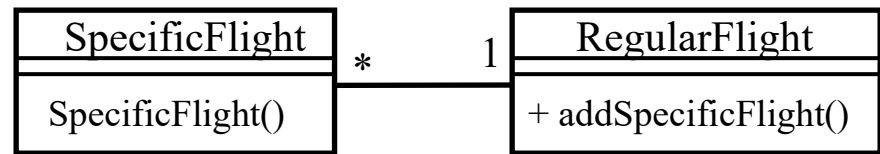


Example: operations for the Airline system



Creating an object and linking it to an existing object (with a bi-directional link)

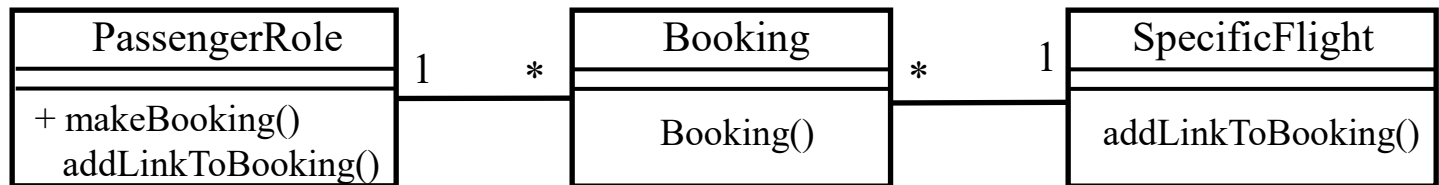
Example: Creating a `SpecificFlight` and linking it to a `RegularFlight`. This responsibility can be accomplished by two operations (the Java implementation is given in section 5.10).



1. (public) The instance of `RegularFlight`
 - calls the constructor of `SpecificFlight`; then, when it is complete, makes a unidirectional link to the new instance of `SpecificFlight` (each instance of `RegularFlight` actually maintains a list of `SpecificFlight` instances; the list is needed to implement the one-to-many relationship).
2. (non-public) Class `SpecificFlight`'s constructor (among its other actions)
 - makes a one-directional link back to the instance of `RegularFlight`.

Creating an association class, given two existing objects

Example: creating an instance of Booking, which will link a SpecificFlight to a PassengerRole. This responsibility can be accomplished in four steps (we want bi-directional associations):



1. (public) The instance of PassengerRole
 - calls the constructor of Booking (operation 2).
2. (non-public) Class Booking's constructor, among its other actions
 - makes a one-directional link back to the instance of PassengerRole
 - makes a one-directional link to the instance of SpecificFlight
 - calls operations 3 and 4.
3. (non-public) The instance of SpecificFlight
 - makes a one-directional link to the instance of Booking.
4. (non-public) The instance of PassengerRole
 - makes a one-directional link to the instance of Booking.

5.10 Implementing Class Diagrams in Java

- Attributes are implemented as instance variables
- Generalizations are implemented using extends
- Interfaces are implemented using implements
- Associations are normally implemented using instance variables
 - Divide each two-way association into two one-way associations
 - so that each associated class has an instance variable representing the other end of the association
 - For a one-way association where the multiplicity at the other end is ‘one’ or ‘optional’
 - declare a variable of that class (a reference)
 - For a one-way association where the multiplicity at the other end is ‘many’:
 - use a collection class implementing List, such as ArrayList, or LinkedList.

Example: SpecificFlight

```
class SpecificFlight
{
    private Calendar date;
    private RegularFlight regularFlight;
    ...
    // Constructor that should only be called from
    // addSpecificFlight
    SpecificFlight( Calendar aDate, RegularFlight aRegularFlight)
    {
        date = aDate;
        regularFlight = aRegularFlight;
    }
}
```

Example: RegularFlight

```
class RegularFlight
{
    private List specificFlights;
    ...
    // Method that has primary responsibility
    public void addSpecificFlight(Calendar aDate)
    {
        SpecificFlight newSpecificFlight;
        newSpecificFlight = new SpecificFlight(aDate, this);
        specificFlights.add(newSpecificFlight);
    }
    ...
}
```

Additional references

[BRJ99] G. Booch, J. Rumbaugh and I. Jacobson. *The Unified Modeling Language user guide*. Addison-Wesley, 1999.

[JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.

[RJB99] J. Rumbaugh, I. Jacobson and G. Booch. *The Unified Modeling Language reference manual*. Addison-Wesley, 1999.